

PROYECTO ACADÉMICO — PROCESAMIENTO DE IMÁGENES

Resumen General, Justificación de Librerías y Análisis Línea por Línea (Ejercicios 1 al 3)

1. Resumen General de Cada Ejercicio

Ejercicio 1: Cálculo de Momentos y Centroides

Objetivo: Analizar tres figuras binarias (*a.png*, *b.png* y *c.png*): a) calcular el área y el centroide de la Figura 1.a; b) obtener el momento de orden $p=2$, $q=3$, su momento central y su momento central normalizado para la Figura 1.b; c) calcular los tres primeros momentos invariantes de Hu para la Figura 1.c.

Mecanismo: Cada imagen se binariza con el método de Otsu (objeto = 1, fondo = 0). Se implementan desde cero, con NumPy, las fórmulas $m_{pq} = \sum \sum x^p y^q f(x,y)$, el momento central μ_{pq} (respecto del centroide) y el normalizado $\eta_{pq} = \mu_{pq} / \mu_{00}^{\gamma}$ con $\gamma = (p+q)/2 + 1$. Todos los resultados se validan contra `cv2.moments()` y `cv2.HuMoments()` de OpenCV.

Ejercicio 2: Histograma de la Imagen "a" con PIL

Objetivo: Obtener y graficar el histograma de la imagen *mono.png* usando exclusivamente la librería PIL/Pillow, tanto en escala de grises como por canal R, G y B.

Mecanismo: Con `Image.histogram()` se obtienen las frecuencias (256 valores en gris y 768 en RGB, segmentados por canal), verificando que la suma coincide con el total de píxeles. `ImageStat.Stat` calcula media, desviación estándar, mediana y extremos. Toda la gráfica (paneles, grilla, barras, curvas, ejes y etiquetas) se dibuja manualmente con `ImageDraw` sobre un lienzo de 6 paneles, sin usar Matplotlib.

Ejercicio 3: Separación de Planos R, G, B y Conversión a Escala de Grises

Objetivo: Separar los tres planos de color de la imagen *flores.png* y convertirla a escala de grises.

Mecanismo: La imagen se carga con OpenCV (que trabaja en BGR) y se convierte a RGB. Con `cv2.split()` se separan los canales, que se muestran como intensidad en gris y como imagen 'teñida' con su color puro, junto con el histograma de cada canal (`cv2.calcHist()`). La conversión a gris se realiza de dos formas: con `cv2.cvtColor(IMG2GRAY)` y manualmente con la fórmula de luminancia ITU-R BT.601 ($Y = 0.299R + 0.587G + 0.114B$), comparando ambos resultados con un mapa de diferencias absolutas.

2. Librerías Utilizadas y Justificación Técnica

Librería / Módulo	Funciones Principales Usadas	Justificación de Uso en el Proyecto
OpenCV (cv2) Ej. 1 y 3	imread(), cvtColor(), threshold() (Otsu), moments(), HuMoments(), resize(), drawMarker(), split(), calcHist(), absdiff(), imwrite()	Librería de visión por computadora optimizada en C++. Permite binarizar automáticamente con Otsu, calcular momentos de referencia para validar la implementación propia, separar canales y convertir espacios de color de forma eficiente. Se tiene en cuenta que OpenCV carga las imágenes en orden BGR .
PIL / Pillow (Image, ImageDraw, ImageFont, ImageStat) Ej. 2	Image.open(), convert(), histogram(), resize(), paste(), rotate(), ImageDraw.rectangle(), rounded_rectangle(), line(), polygon(), text(), ImageStat.Stat	El enunciado del Ejercicio 2 exige usar PIL. histogram() devuelve directamente las frecuencias por nivel, ImageStat entrega las estadísticas por banda e ImageDraw permite construir la gráfica completa píxel a píxel, con transparencias (RGBA) y fuentes TrueType.
NumPy (numpy) Ej. 1 y 3	indices(), sum(), nonzero(), count_nonzero(), mean(), zeros(), clip(), round(), astype()	Permite expresar las fórmulas de momentos como operaciones vectorizadas sobre mallas de coordenadas (x, y), sin bucles explícitos. También se usa para la conversión manual a gris en coma flotante y para construir las imágenes 'teñidas' de cada canal.
Matplotlib (pyplot) Ej. 1 y 3	subplots(), imshow(), text(), plot(), fill_between(), colorbar(), tight_layout(), savefig(), show()	Generación de figuras comparativas multipanel: imágenes con centroides marcados, paneles de texto con los resultados numéricos, histogramas por canal y mapas de diferencia con barra de color. Exporta a PNG a 150 DPI.
os	path.dirname(), path.abspath(), path.join(), path.isfile(), path.basename()	Construye rutas relativas a la ubicación del script, garantizando que el programa funcione en cualquier sistema operativo y desde cualquier directorio de trabajo.

3.1 Ejercicio 1 — Cálculo de Momentos y Centroides

Archivo fuente: `ejercicio_1.py` (Código limpio sin comentarios)

Bloque 1: Importación de Librerías y Definición de Rutas

```
3 | import os
5 | import cv2
6 | import numpy as np
7 | import matplotlib.pyplot as plt
9 | directorio = os.path.dirname(os.path.abspath(__file__))
14 | RUTA_FIGURA_1A = os.path.join(directorio, "a.png")
15 | RUTA_FIGURA_1B = os.path.join(directorio, "b.png")
16 | RUTA_FIGURA_1C = os.path.join(directorio, "c.png")
17 | RUTA_SALIDA = os.path.join(directorio, "ejercicio_1_resultado.png")
```

Explicación línea por línea:

- **Línea 3:** Importa el módulo `os` para construir rutas de archivo independientes del sistema operativo.
- **Línea 5-7:** Importa OpenCV (`cv2`) para lectura, binarización y validación de momentos; NumPy para el cálculo vectorizado; y `pyplot` de Matplotlib para la figura de resultados.
- **Línea 9:** Obtiene la carpeta donde se encuentra el script, de modo que las imágenes se localicen sin importar desde dónde se ejecute.
- **Línea 14-16:** Rutas absolutas de las tres figuras binarias del enunciado: 1.a, 1.b y 1.c.
- **Línea 17:** Ruta del archivo PNG donde se guardará la figura final con todos los resultados.

Bloque 2: Carga y Binarización Automática de la Imagen (Otsu)

```
23 | def cargar_imagen_binaria(ruta):
25 |     img_bgr = cv2.imread(ruta, cv2.IMREAD_COLOR)
26 |     if img_bgr is None:
27 |         raise FileNotFoundError(f"No se pudo cargar la imagen: {ruta}")
29 |     gris = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
30 |     _, binaria = cv2.threshold(gris, 0, 1, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)
32 |     if binaria.mean() > 0.5:
33 |         binaria = 1 - binaria
35 |     return img_bgr, binaria.astype(np.uint8)
```

Explicación línea por línea:

- **Línea 23:** Define la función que transforma cualquier figura en una matriz binaria $f(x,y)$ con valores 0 (fondo) y 1 (objeto).
- **Línea 25-27:** Lee la imagen en color (OpenCV la entrega en orden BGR). Si `imread` devuelve `None` el archivo no existe o está dañado, y se lanza una excepción descriptiva.
- **Línea 29:** Convierte la imagen a escala de grises, requisito para la umbralización.
- **Línea 30:** Aplica el umbral de **Otsu**, que elige automáticamente el valor de corte óptimo. Con `THRESH_BINARY_INV` y valor máximo 1, los píxeles oscuros (el objeto) quedan en 1 y el fondo claro en 0.
- **Línea 32-33:** Medida de seguridad: si más de la mitad de los píxeles quedaron en 1, se asume que se invirtió objeto/fondo y se corrige con `1 - binaria`.
- **Línea 35:** Devuelve la imagen original (para dibujar sobre ella) y la máscara binaria como enteros de 8 bits.

Bloque 3: Mallas de Coordenadas, Momento Espacial y Centroide

```

38 | def obtener_coordenadas(binaria):
39 |     filas, columnas = np.indices(binaria.shape, dtype=np.float64)
40 |     return columnas, filas
41 | def momento_especial(binaria, p, q):
42 |     x, y = obtener_coordenadas(binaria)
43 |     f = binaria.astype(np.float64)
44 |     return float(np.sum((x ** p) * (y ** q) * f))
45 | def centroide_por_momentos(binaria):
46 |     m00 = momento_especial(binaria, 0, 0)
47 |     if m00 == 0:
48 |         raise ValueError("La imagen no contiene ningún objeto (m00 = 0).")
49 |     return momento_especial(binaria, 1, 0) / m00, momento_especial(binaria, 0, 1) / m00

```

Explicación línea por línea:

- **Línea 38-40:** `np.indices` genera dos matrices del mismo tamaño que la imagen: una con el número de fila (coordenada y) y otra con el de columna (coordenada x) de cada píxel. Se usan en coma flotante para evitar desbordamientos al elevar a potencias.
- **Línea 43-46:** Implementa el momento espacial de orden $(p+q)$: $m_{pq} = \sum \sum x^p y^q f(x,y)$. La multiplicación elemento a elemento y `np.sum` sustituyen a los dos bucles anidados de la fórmula.
- **Línea 49-50:** m_{00} es la suma de todos los píxeles del objeto, es decir, su **área**.
- **Línea 51-52:** Evita una división por cero si la imagen no contiene ningún objeto.
- **Línea 53:** Devuelve el centroide $(x_c, y_c) = (m_{10}/m_{00}, m_{01}/m_{00})$, el 'centro de masa' de la figura.

Bloque 4: Momento Central y Momento Central Normalizado

```

56 | def momento_central(binaria, p, q):
57 |     x, y = obtener_coordenadas(binaria)
58 |     f = binaria.astype(np.float64)
59 |     x_c, y_c = centroide_por_momentos(binaria)
60 |     return float(np.sum(((x - x_c) ** p) * ((y - y_c) ** q) * f))
61 | def momento_central_normalizado(binaria, p, q):
62 |     mu00 = momento_central(binaria, 0, 0)
63 |     gamma = (p + q) / 2.0 + 1.0
64 |     return momento_central(binaria, p, q) / (mu00 ** gamma)

```

Explicación línea por línea:

- **Línea 56-59:** Obtiene las coordenadas, la imagen en flotante y el centroide del objeto.
- **Línea 60:** Momento central: $\mu_{pq} = \sum \sum (x - x_c)^p (y - y_c)^q f(x,y)$. Al medir respecto del centroide, el resultado es **invariante a la traslación**.
- **Línea 63-64:** μ_{00} coincide con m_{00} (el área) y se usa como factor de escala.
- **Línea 65:** Exponente de normalización $\gamma = (p+q)/2 + 1$, definido por la teoría de momentos.
- **Línea 66:** Momento central normalizado: $\eta_{pq} = \mu_{pq} / \mu_{00}^{\gamma}$. La división hace al descriptor **invariante a la escala** además de a la traslación.

Bloque 5: Marcado del Centroide sobre la Imagen Ampliada

```
69 | def dibujar_cruz(img_bgr, x, y, escala, color, tipo=cv2.MARKER_CROSS, tam=40, grosor=2):
70 |     punto = (int(round((x + 0.5) * escala)), int(round((y + 0.5) * escala)))
71 |     cv2.drawMarker(img_bgr, punto, color, markerType=tipo,
72 |                   markerSize=tam, thickness=grosor, line_type=cv2.LINE_AA)
```

Explicación línea por línea:

- **Línea 69:** Función auxiliar para dibujar un marcador en la posición del centroide, con color, forma, tamaño y grosor configurables.
- **Línea 70:** Convierte la coordenada del píxel original a la imagen ampliada. Se suma 0.5 para ubicar la marca en el **centro** del píxel (cada píxel original ocupa un cuadrado de *escala* x *escala*).
- **Línea 71-72:** cv2.drawMarker dibuja la cruz con antialiasing (LINE_AA) para bordes suaves.

Bloque 6: Inciso a) Área y Centroide de la Figura 1.a

```

78 | def analizar_figura_1a(ruta, escala=5):
79 |     img_bgr, binaria = cargar_imagen_binaria(ruta)
82 |     area = int(np.count_nonzero(binaria))
85 |     ys, xs = np.nonzero(binaria)
86 |     cx_geo, cy_geo = float(xs.mean()), float(ys.mean())
89 |     m00 = momento_espacial(binaria, 0, 0)
90 |     m10 = momento_espacial(binaria, 1, 0)
91 |     m01 = momento_espacial(binaria, 0, 1)
92 |     cx_mom, cy_mom = m10 / m00, m01 / m00
95 |     M = cv2.moments(binaria, binaryImage=True)
96 |     cx_cv, cy_cv = M["m10"] / M["m00"], M["m01"] / M["m00"]
99 |     img_marcada = cv2.resize(img_bgr, None, fx=escala, fy=escala,
100 |                             interpolation=cv2.INTER_NEAREST)
102 |     dibujar_cruz(img_marcada, cx_geo, cy_geo, escala, (255, 80, 0),
103 |                  cv2.MARKER_CROSS, tam=50, grosor=3)
104 |     dibujar_cruz(img_marcada, cx_mom, cy_mom, escala, (0, 0, 0),
105 |                  cv2.MARKER_TILTED_CROSS, tam=30, grosor=2)
107 |     print("\n--- a) Figura 1.a: Área y Centroides ---")
108 |     print(f" Área (píxeles del objeto):      {area}")
114 |     return {
115 |         "area": area,
116 |         "centroide": (cx_geo, cy_geo),
117 |         "centroide_momentos": (cx_mom, cy_mom),
120 |         "imagen_marcada": cv2.cvtColor(img_marcada, cv2.COLOR_BGR2RGB),
122 |     }

```

Explicación línea por línea:

- **Línea 78-79:** Carga la Figura 1.a y obtiene su versión binaria. El parámetro `escala=5` define cuánto se ampliará para visualizarla.
- **Línea 82:** Área = cantidad de píxeles con valor 1, contados con `np.count_nonzero`.
- **Línea 85-86: Centroide geométrico:** `np.nonzero` devuelve las coordenadas (filas, columnas) de los píxeles del objeto; su promedio es el centro geométrico.
- **Línea 89-92: Centroide por momentos:** se calculan m_{00} , m_{10} y m_{01} con la función propia y se aplica $x_c = m_{10}/m_{00}$, $y_c = m_{01}/m_{00}$. Ambos métodos deben coincidir.
- **Línea 95-96:** Validación cruzada con `cv2.moments` de OpenCV (con `binaryImage=True` trata todo valor distinto de 0 como 1).
- **Línea 99-100:** Amplía la imagen 5 veces con `INTER_NEAREST` (vecino más cercano) para conservar los bordes nítidos de cada píxel y poder ver bien las marcas.
- **Línea 102-105:** Dibuja el centroide geométrico con una cruz azul (`BGR = (255, 80, 0)`) y el centroide por momentos con un aspa negra más pequeña; si se superponen, se confirma que ambos coinciden.
- **Línea 107-112:** Imprime en consola el área, los momentos y los tres centroides con 4 decimales.
- **Línea 114-122:** Devuelve un diccionario con todos los resultados. La imagen marcada se convierte de BGR a RGB porque Matplotlib espera el orden RGB.

Bloque 7: Inciso b) Momentos de Orden $p=2$, $q=3$ (Figura 1.b)

```

128 | def momentos_figura_1b(ruta, p=2, q=3, escala=5):
129 |     img_bgr, binaria = cargar_imagen_binaria(ruta)
131 |     m_pq = momento_espacial(binaria, p, q)
132 |     mu_pq = momento_central(binaria, p, q)
133 |     eta_pq = momento_central_normalizado(binaria, p, q)
134 |     cx, cy = centroide_por_momentos(binaria)
137 |     M = cv2.moments(binaria, binaryImage=True)
138 |     validacion = {
139 |         "m21": (momento_espacial(binaria, 2, 1), M["m21"]),
140 |         "mu21": (momento_central(binaria, 2, 1), M["mu21"]),
141 |         "nu21": (momento_central_normalizado(binaria, 2, 1), M["nu21"]),
142 |     }
144 |     img_marcada = cv2.resize(img_bgr, None, fx=escala, fy=escala,
145 |                             interpolation=cv2.INTER_NEAREST)
146 |     dibujar_cruz(img_marcada, cx, cy, escala, (0, 0, 255), tam=40, grosor=3)
148 |     print(f"\n--- b) Figura 1.b: Momentos de orden p={p}, q={q} ---")
154 |     for nombre, (propio, opencv) in validacion.items():
155 |         print(f"    {nombre:<5} propio = {propio: .6e} OpenCV = {opencv: .6e}")
157 |     return {
158 |         "p": p, "q": q,
159 |         "m": m_pq, "mu": mu_pq, "eta": eta_pq,
164 |     }

```

Explicación línea por línea:

- **Línea 128-129:** Carga la Figura 1.b. Los parámetros por defecto $p=2$ y $q=3$ corresponden al orden pedido en el enunciado (orden total 5).
- **Línea 131-133:** Calcula los tres descriptores solicitados: momento espacial m_{23} , momento central μ_{23} y momento central normalizado η_{23} .
- **Línea 134:** Calcula el centroide, necesario para el momento central y para marcarlo en la imagen.
- **Línea 137-142:** cv2.moments solo calcula momentos hasta orden 3, por lo que m_{23} no puede compararse directamente. Por eso se valida la implementación propia con el orden (2,1): si m_{21} , μ_{21} y ν_{21} coinciden con OpenCV, las mismas funciones son correctas para (2,3).
- **Línea 144-146:** Amplía la imagen y marca el centroide con una cruz roja (BGR = (0, 0, 255)).
- **Línea 148-155:** Imprime los momentos en notación científica (.6e), ya que los momentos de orden alto alcanzan valores muy grandes, y muestra la tabla de validación.
- **Línea 157-164:** Devuelve los resultados, la validación y la imagen marcada en RGB.

Bloque 8: Inciso c) Momentos Invariantes de Hu (Figura 1.c)

```

170 | def momentos_hu_figura_1c(ruta, escala=3):
171 |     img_bgr, binaria = cargar_imagen_binaria(ruta)
173 |     eta = {"eta{p}{q}": momento_central_normalizado(binaria, p, q)
174 |           for p, q in [(2, 0), (0, 2), (1, 1), (3, 0), (1, 2), (2, 1), (0, 3)]}
176 |     h1 = eta["eta20"] + eta["eta02"]
177 |     h2 = (eta["eta20"] - eta["eta02"]) ** 2 + 4 * eta["eta11"] ** 2
178 |     h3 = (eta["eta30"] - 3 * eta["eta12"]) ** 2 + (3 * eta["eta21"] - eta["eta03"]) ** 2
180 |     hu_cv = cv2.HuMoments(cv2.moments(binaria, binaryImage=True)).flatten()[3]
182 |     img_ampliada = cv2.resize(img_bgr, None, fx=escala, fy=escala,
183 |                               interpolation=cv2.INTER_NEAREST)
185 |     print("\n--- c) Figura 1.c: Momentos invariantes de Hu ---")
186 |     for i, (propio, opencv) in enumerate(zip((h1, h2, h3), hu_cv), start=1):
187 |         print(f"  H{i}: propio = {propio:.6e}  OpenCV = {opencv:.6e}")
189 |     return {
190 |         "hu": (h1, h2, h3),
191 |         "hu_opencv": tuple(float(v) for v in hu_cv),
195 |     }

```

Explicación línea por línea:

- **Línea 170-171:** Carga y binariza la Figura 1.c.
- **Línea 173-174:** Comprensión de diccionario que calcula los 7 momentos centrales normalizados de orden 2 y 3 necesarios para las fórmulas de Hu, con claves como 'eta20', 'eta11', etc.
- **Línea 176:** $H1 = \eta_{20} + \eta_{02}$: mide la dispersión total del objeto respecto de su centroide.
- **Línea 177:** $H2 = (\eta_{20} - \eta_{02})^2 + 4 \eta_{11}^2$: mide la elongación (alargamiento) de la figura.
- **Línea 178:** $H3 = (\eta_{30} - 3 \eta_{12})^2 + (3 \eta_{21} - \eta_{03})^2$: mide la asimetría de la figura. Los tres son invariantes a traslación, escala y rotación.
- **Línea 180:** Calcula los 7 momentos de Hu con OpenCV, los aplan a un vector 1D y se queda con los 3 primeros para compararlos.
- **Línea 182-183:** Amplía la figura x3 para su visualización.
- **Línea 185-187:** Imprime cada H_i propio junto al valor de OpenCV para verificar la implementación.
- **Línea 189-195:** Devuelve los momentos de Hu propios y de OpenCV, los eta utilizados y la imagen en RGB.

Bloque 9: Panel de Texto y Configuración de la Figura

```

201 | def _panel_texto(ax, titulo, texto):
202 |     ax.axis("off")
203 |     ax.set_title(titulo, fontsize=12, fontweight="bold")
204 |     ax.text(0.02, 0.95, texto, transform=ax.transAxes, va="top", ha="left",
205 |           family="monospace", fontsize=10.5,
206 |           bbox=dict(boxstyle="round,pad=0.8", facecolor="#f4f6fb", edgecolor="#c9d1e3"))
207 |
210 | def graficar_resultados(res_a, res_b, res_c, ruta_salida=None, mostrar=True):
211 |     fig, axes = plt.subplots(3, 2, figsize=(15, 15),
212 |                             gridspec_kw={"width_ratios": [1.1, 1]})
213 |     fig.suptitle("Ejercicio 1: Momentos y Centroides", fontsize=16, fontweight="bold")

```

Explicación línea por línea:

- **Línea 201-204:** Función auxiliar que convierte un eje de Matplotlib en un panel de texto: oculta los ejes y coloca un título en negrita.
- **Línea 205-207:** Escribe el texto en la esquina superior izquierda (coordenadas relativas al eje con transAxes), con fuente monoespaciada para alinear columnas y dentro de una caja redondeada de fondo gris azulado.
- **Línea 210-212:** Crea una cuadrícula de 3 filas (una por inciso) x 2 columnas (imagen | resultados), donde la columna de la imagen es un 10% más ancha.
- **Línea 213:** Título general de la figura.

Bloque 10: Composición de los Tres Incisos y Guardado

```

216 |     axes[0, 0].imshow(res_a["imagen_marcada"])
217 |     _panel_texto(axes[0, 1], "a) Área y Centroide", (
218 |         axes[1, 0].imshow(res_b["imagen_marcada"])
219 |         texto_val = "\n".join(f" {k:<5} propio={v[0]: .4e}   cv2={v[1]: .4e}"
220 |                               for k, v in res_b["validacion"].items())
221 |     _panel_texto(axes[1, 1], f"b) Momentos de orden p={p}, q={q}", (
222 |         axes[2, 0].imshow(res_c["imagen"])
223 |         texto_hu = "\n".join(
224 |             f"H{i} = {h:.6e}   (cv2: {hc:.6e})"
225 |             for i, (h, hc) in enumerate(zip(res_c["hu"], res_c["hu_opencv"]), start=1))
226 |         _panel_texto(axes[2, 1], "c) Momentos invariantes de Hu", (
227 |             plt.tight_layout(rect=[0, 0, 1, 0.97])
228 |             if ruta_salida:
229 |                 plt.savefig(ruta_salida, dpi=150, bbox_inches="tight")
230 |             if mostrar:
231 |                 plt.show()
232 |             return fig

```

Explicación línea por línea:

- **Línea 216-230:** Fila 1 (inciso a): muestra la Figura 1.a con los dos centroides marcados y, a la derecha, el área, los momentos m_{00} , m_{10} , m_{01} y los centroides geométrico, por momentos y de OpenCV.
- **Línea 233-245:** Fila 2 (inciso b): muestra la Figura 1.b con su centroide y un panel con m_{23} , μ_{23} , η_{23} y la tabla de validación de orden (2,1) generada con join.
- **Línea 248-257:** Fila 3 (inciso c): muestra la Figura 1.c y un panel con H1, H2 y H3 (propios y de OpenCV) más los momentos normalizados usados.
- **Línea 259:** Ajusta automáticamente los espacios entre subplots, reservando el 3% superior para el título.
- **Línea 260-265:** Si se indicó una ruta, guarda la figura a 150 DPI; si mostrar es verdadero, abre la ventana interactiva. Devuelve la figura para posible reutilización.

Bloque 11: Programa Principal

```
271 | if __name__ == "__main__":
272 |     print("=" * 60)
273 |     print(" EJERCICIO 1: Cálculo de Momentos y Centroides")
276 |     resultado_a = analizar_figura_1a(RUTA_FIGURA_1A)
277 |     resultado_b = momentos_figura_1b(RUTA_FIGURA_1B, p=2, q=3)
278 |     resultado_c = momentos_hu_figura_1c(RUTA_FIGURA_1C)
280 |     graficar_resultados(resultado_a, resultado_b, resultado_c, ruta_salida=RUTA_SALIDA)
282 |     print("\n" + "=" * 60)
283 |     print(" Ejercicio 1 completado exitosamente.")
```

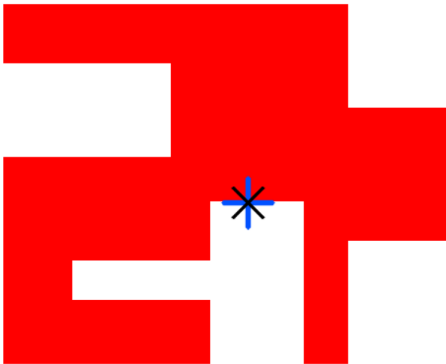
Explicación línea por línea:

- **Línea 271:** El bloque solo se ejecuta al correr el archivo directamente, no al importarlo como módulo (lo que permite reutilizar sus funciones).
- **Línea 272-274:** Imprime el encabezado del ejercicio.
- **Línea 276-278:** Ejecuta en orden los tres incisos: a) área y centroide, b) momentos de orden (2,3) y c) momentos de Hu.
- **Línea 280:** Genera y guarda la figura comparativa con los tres resultados.
- **Línea 282-284:** Imprime el mensaje de finalización.

Resultado obtenido

Ejercicio 1: Momentos y Centroides

a) Figura 1.a - Centroides
Cruz azul: geométrico | Aspa negra: por momentos



a) Área y Centroide

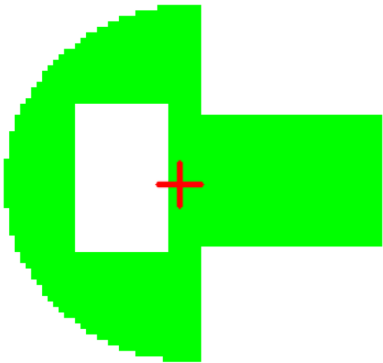
```
Área (píxeles)      = 3179
m00 = 3179
m10 = 162203
m01 = 132766

Centroide geométrico:
(x, y) = (51.023, 41.763)

Centroide por momentos (m10/m00, m01/m00):
(x, y) = (51.023, 41.763)

Verificación cv2.moments:
(x, y) = (51.023, 41.763)
```

b) Figura 1.b - Centroide (cruz roja)



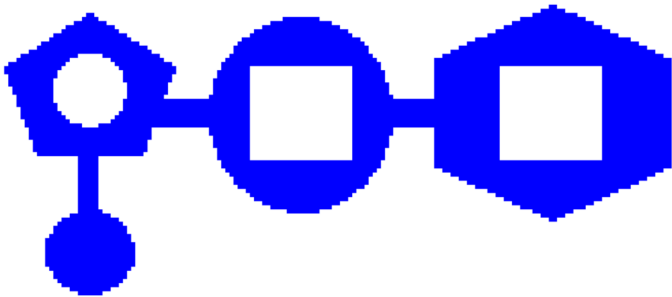
b) Momentos de orden p=2, q=3

```
Centroide (x_c, y_c) = (33.508, 34.014)

Momento de orden      m(2,3) = 1.685249e+11
Momento central      mu(2,3) = -2.178926e+07
Momento central normalizado eta(2,3) = -4.418762e-05

Validación de la implementación (orden 2,1):
m21 propio= 1.0818e+08 cv2= 1.0818e+08
mu21 propio=-1.5149e+05 cv2=-1.5149e+05
nu21 propio=-6.7341e-04 cv2=-6.7341e-04
```

c) Figura 1.c



c) Momentos invariantes de Hu

```
H1 = 6.399982e-01 (cv2: 6.399982e-01)
H2 = 2.770505e-01 (cv2: 2.770505e-01)
H3 = 1.768381e-03 (cv2: 1.768381e-03)

Momentos centrales normalizados usados:
eta20 = 5.809963e-01
eta02 = 5.900190e-02
eta11 = -3.380944e-02
eta30 = -1.006028e-02
eta12 = -1.467333e-02
eta21 = 1.032660e-02
eta03 = 6.178207e-03
```

Figura 1. Resultados de los incisos a), b) y c) del Ejercicio 1 (ejercicio_1_resultado.png)

3.2 Ejercicio 2 — Histograma de la Imagen "a" con PIL

Archivo fuente: ejercicio_2.py (Código limpio sin comentarios)

Bloque 1: Importaciones, Rutas y Paleta de Colores

```
1 | import os
3 | from PIL import Image, ImageDraw, ImageFont, ImageStat
5 | directorio = os.path.dirname(os.path.abspath(__file__))
10 | RUTA_IMAGEN_A = os.path.join(directorio, "mono.png")
11 | RUTA_SALIDA = os.path.join(directorio, "ejercicio_2_resultado.png")
14 | COLOR_FONDO = (243, 245, 250)
15 | COLOR_PANEL = (255, 255, 255)
16 | COLOR_BORDE = (214, 219, 230)
17 | COLOR_TEXTO = (35, 40, 55)
18 | COLOR_SECUNDARIO = (110, 118, 135)
19 | COLOR_GRILLA = (232, 235, 242)
20 | COLORES_CANAL = {
21 |     "Gris": (70, 75, 90),
22 |     "R": (226, 61, 74),
23 |     "G": (46, 168, 96),
24 |     "B": (52, 110, 230),
25 | }
```

Explicación línea por línea:

- **Línea 1:** Importa `os` para el manejo de rutas.
- **Línea 3:** Importa los cuatro módulos de Pillow utilizados: `Image` (carga y lienzo), `ImageDraw` (dibujo 2D), `ImageFont` (tipografías) e `ImageStat` (estadísticas). No se usa Matplotlib: todo el gráfico se construye con PIL.
- **Línea 5:** Directorio del script, base para las rutas relativas.
- **Línea 10-11:** Ruta de la imagen de entrada "a" (*mono.png*) y del PNG de salida.
- **Línea 14-19:** Paleta de colores RGB de la interfaz: fondo general, paneles, bordes, texto principal, texto secundario y líneas de la grilla.
- **Línea 20-25:** Diccionario con el color asignado a cada histograma: gris oscuro, rojo, verde y azul.

Bloque 2: Carga de la Imagen y Cálculo de los Histogramas

```
31 | def cargar_imagen(ruta):
32 |     if not os.path.isfile(ruta):
33 |         raise FileNotFoundError(f"No se encontró la imagen: {ruta}")
34 |     return Image.open(ruta).convert("RGB")
37 | def calcular_histogramas(img_rgb):
38 |     hist_gris = img_rgb.convert("L").histogram()
39 |     hist_rgb = img_rgb.histogram()
41 |     histogramas = {
42 |         "Gris": hist_gris,
43 |         "R": hist_rgb[0:256],
44 |         "G": hist_rgb[256:512],
45 |         "B": hist_rgb[512:768],
46 |     }
49 |     total = img_rgb.width * img_rgb.height
50 |     for nombre, hist in histogramas.items():
51 |         assert sum(hist) == total, f"El histograma {nombre} no suma {total}"
53 |     return histogramas
```

Explicación línea por línea:

- **Línea 31-34:** Verifica que el archivo exista y lo abre forzando el modo RGB (así funciona igual con imágenes en paleta, RGBA o escala de grises).
- **Línea 38:** Convierte a escala de grises (modo 'L', luminancia ITU-R 601-2) y obtiene su histograma: una lista de 256 frecuencias, una por nivel de intensidad.
- **Línea 39:** En una imagen RGB, histogram() devuelve una única lista de 768 valores: los 256 niveles de R, seguidos por los de G y luego los de B.
- **Línea 41-46:** Organiza los cuatro histogramas en un diccionario, separando la lista RGB en tramos [0:256], [256:512] y [512:768].
- **Línea 49-51:** Control de calidad: la suma de las frecuencias de cada histograma debe ser exactamente el total de píxeles (ancho x alto); si no, assert detiene el programa.
- **Línea 53:** Devuelve el diccionario de histogramas.

Bloque 3: Estadísticas Descriptivas con ImageStat

```

56 | def calcular_estadisticas(img_rgb):
57 |     est_gris = ImageStat.Stat(img_rgb.convert("L"))
58 |     est_rgb = ImageStat.Stat(img_rgb)
59 |     estadisticas = {}
60 |     for i, canal in enumerate(["Gris", "R", "G", "B"]):
61 |         est, k = (est_gris, 0) if canal == "Gris" else (est_rgb, i - 1)
62 |         estadisticas[canal] = {
63 |             "media": est.mean[k],
64 |             "desv": est.stddev[k],
65 |             "mediana": est.median[k],
66 |             "min": est.extrema[k][0],
67 |             "max": est.extrema[k][1],
68 |         }
69 |     return estadisticas
70 |

```

Explicación línea por línea:

- **Línea 57-58:** ImageStat.Stat calcula estadísticas por banda: una para la imagen en gris (1 banda) y otra para la RGB (3 bandas).
- **Línea 61-62:** Recorre los cuatro canales; para 'Gris' usa la banda 0 de est_gris y para R, G, B usa las bandas 0, 1 y 2 de est_rgb (índice i - 1).
- **Línea 63-69:** Guarda la media (brillo promedio), la desviación estándar (contraste), la mediana y los valores mínimo y máximo (extrema) de cada canal.
- **Línea 70:** Devuelve el diccionario de estadísticas, usado en los subtítulos de los paneles y en la tabla de consola.

Bloque 4: Carga de Fuentes Tipográficas

```

76 | def cargar_fuente(tamano, negrita=False):
77 |     candidatos = (["arialbd.ttf", "segoeui.ttf", "DejaVuSans-Bold.ttf"] if negrita
78 |                   else ["arial.ttf", "segoeui.ttf", "DejaVuSans.ttf"])
79 |     for nombre in candidatos:
80 |         try:
81 |             return ImageFont.truetype(nombre, tamano)
82 |         except OSError:
83 |             continue
84 |     try:
85 |         return ImageFont.load_default(size=tamano)
86 |     except TypeError:
87 |         return ImageFont.load_default()
88 | FUENTE_TITULO = cargar_fuente(28, negrita=True)
89 | FUENTE_PANEL = cargar_fuente(17, negrita=True)
90 | FUENTE_TEXTO = cargar_fuente(13)
91 | FUENTE_TICKS = cargar_fuente(12)
92 |

```

Explicación línea por línea:

- **Línea 76-79:** Lista de fuentes TrueType candidatas (Arial y Segoe UI en Windows, DejaVu en Linux), en versión negrita o normal según el parámetro.
- **Línea 80-84:** Intenta cargar cada fuente en orden; si no está instalada, truetype lanza OSError y se prueba la siguiente.
- **Línea 85-88:** Si ninguna existe, usa la fuente por defecto de PIL. Las versiones antiguas de Pillow no aceptan el parámetro size (TypeError), por lo que se carga sin él.
- **Línea 91-94:** Crea las cuatro fuentes globales: título principal (28 px), títulos de panel (17 px), texto general (13 px) y números de los ejes (12 px).

Bloque 5: Utilidades: Formato Numérico y Texto Vertical

```

97 | def _formato_numero(valor):
98 |     if valor >= 1_000_000:
99 |         return f"{valor / 1_000_000:.1f}M"
100 |     if valor >= 1_000:
101 |         return f"{valor / 1_000:.1f}k"
102 |     return f"{int(valor)}"
103 |
104 | def _texto_vertical(lienzo, texto, centro, fuente, color):
105 |     caja = fuente.getbbox(texto)
106 |     ancho, alto = caja[2] - caja[0] + 4, caja[3] - caja[1] + 6
107 |     capa = Image.new("RGBA", (ancho, alto), (0, 0, 0, 0))
108 |     ImageDraw.Draw(capa).text((2 - caja[0], 2 - caja[1]), texto, font=fuente, fill=color)
109 |     capa = capa.rotate(90, expand=True)
110 |     lienzo.paste(capa, (int(centro[0] - capa.width / 2), int(centro[1] - capa.height / 2)), capa)

```

Explicación línea por línea:

- **Línea 97-103:** Abrevia las frecuencias grandes del eje Y: por ejemplo, 12500 se muestra como "12.5k" y 2300000 como "2.3M", para que las etiquetas no ocupen demasiado espacio.
- **Línea 106-109:** PIL no puede escribir texto rotado directamente; por eso se mide el texto con `getbbox` para conocer su tamaño exacto.
- **Línea 110-111:** Crea una capa RGBA transparente del tamaño del texto y escribe el texto en ella.
- **Línea 112:** Rota la capa 90° en sentido antihorario; `expand=True` ajusta el tamaño para no recortar el texto.
- **Línea 113:** Pega la capa centrada en la posición indicada, usando la propia capa como máscara para respetar la transparencia. Se usa para la etiqueta del eje Y.

Bloque 6: Panel Base con Sombra y Bordes Redondeados

```

116 | def dibujar_panel(lienzo, caja, titulo, subtítulo=None):
117 |     draw = ImageDraw.Draw(lienzo, "RGBA")
118 |     x0, y0, x1, _ = caja
119 |     draw.rounded_rectangle((x0 + 3, y0 + 4, caja[2] + 3, caja[3] + 4), radius=14, fill=(0, 0, 0, 18))
120 |     draw.rounded_rectangle(caja, radius=14, fill=COLOR_PANEL, outline=COLOR BORDE, width=1)
121 |     draw.text((x0 + 20, y0 + 14), titulo, font=FUENTE_PANEL, fill=COLOR_TEXTO)
122 |     if subtítulo:
123 |         ancho_sub = draw.textlength(subtítulo, font=FUENTE_TEXTO)
124 |         draw.text((x1 - 20 - ancho_sub, y0 + 17), subtítulo, font=FUENTE_TEXTO, fill=COLOR_SECUNDARIO)

```

Explicación línea por línea:

- **Línea 116-119:** Crea un objeto de dibujo en modo RGBA (permite colores semitransparentes) y desempaqueta las coordenadas de la caja del panel.
- **Línea 121:** Dibuja una **sombra**: un rectángulo redondeado negro con opacidad 18/255, desplazado 3 px a la derecha y 4 px hacia abajo.
- **Línea 122:** Dibuja el panel blanco con esquinas redondeadas (radio 14) y un borde gris claro.
- **Línea 123:** Escribe el título del panel en la esquina superior izquierda.
- **Línea 124-126:** Si hay subtítulo (las estadísticas del canal), mide su ancho con `textlength` y lo alinea a la derecha del panel.

Bloque 7: Dibujo del Histograma (I): Área de Gráfico y Grilla

```

129 | def dibujar_histograma(lienzo, caja, series, titulo, subtitulo=None, modo="barras"):
130 |     dibujar_panel(lienzo, caja, titulo, subtitulo)
131 |     draw = ImageDraw.Draw(lienzo, "RGBA")
133 |     x0, y0, x1, y1 = caja
134 |     gx0, gy0, gx1, gy1 = x0 + 78, y0 + 52, x1 - 24, y1 - 62
135 |     ancho, alto = gx1 - gx0, gy1 - gy0
136 |     max_valor = max(max(h) for h, _ in series) or 1
139 |     for i in range(5):
140 |         y = gy1 - alto * i / 4
141 |         draw.line([(gx0, y), (gx1, y)], fill=COLOR_GRILLA, width=1)
142 |         etiqueta = _formato_numero(max_valor * i / 4)
143 |         tw = draw.textlength(etiqueta, font=FUENTE_TICKS)
144 |         draw.text((gx0 - 10 - tw, y - 7), etiqueta, font=FUENTE_TICKS, fill=COLOR_SECUNDARIO)

```

Explicación línea por línea:

- **Línea 129-131:** Función principal de graficado. Recibe una lista de series (pares histograma-color) y el modo: 'barras' para un canal o 'línea' para superponer varios. Primero dibuja el panel base.
- **Línea 133-135:** Define el área útil del gráfico dentro del panel, dejando márgenes para las etiquetas del eje Y (78 px), el título (52 px) y el eje X (62 px).
- **Línea 136:** Frecuencia máxima entre todas las series, usada para escalar verticalmente. El or 1 evita dividir por cero en una imagen vacía.
- **Línea 139-141:** Dibuja 5 líneas horizontales de grilla (0%, 25%, 50%, 75% y 100% del máximo). En imagen, el eje Y crece hacia abajo, por eso se resta desde gy1.
- **Línea 142-144:** Escribe el valor de frecuencia de cada línea, alineado a la derecha del eje Y.

Bloque 8: Dibujo del Histograma (II): Barras o Curvas Rellenas

```

147 |     paso = ancho / 256
148 |     for hist, color in series:
149 |         if modo == "barras":
150 |             for nivel, valor in enumerate(hist):
151 |                 if valor == 0:
152 |                     continue
153 |                 bx0 = gx0 + nivel * paso
154 |                 by = gy1 - alto * valor / max_valor
155 |                 draw.rectangle([bx0, by, bx0 + max(paso - 0.6, 1), gy1], fill=color + (235,))
156 |             else:
157 |                 puntos = [(gx0 + ancho * n / 255, gy1 - alto * v / max_valor) for n, v in enumerate(hist)]
158 |                 draw.polygon([(gx0, gy1)] + puntos + [(gx1, gy1)], fill=color + (55,))
159 |                 draw.line(puntos, fill=color + (255,), width=2, joint="curve")

```

Explicación línea por línea:

- **Línea 147:** Ancho horizontal asignado a cada uno de los 256 niveles de intensidad.
- **Línea 148-152:** Recorre cada serie. En modo barras, recorre los 256 niveles y omite los que tienen frecuencia 0.
- **Línea 153-154:** Calcula la posición x de la barra según su nivel y la altura proporcional a su frecuencia respecto del máximo.
- **Línea 155:** Dibuja la barra como un rectángulo desde la altura calculada hasta la base, dejando una pequeña separación (0.6 px) entre barras y con opacidad 235/255.
- **Línea 157:** En modo línea, convierte cada par (nivel, frecuencia) en un punto (x, y) del gráfico.
- **Línea 158:** Cierra la curva con las esquinas inferiores y la rellena con el color del canal muy transparente (55/255), de modo que las tres curvas RGB se vean superpuestas.
- **Línea 159:** Traza la curva encima con trazo opaco de 2 px y uniones suavizadas (joint='curve').

Bloque 9: Dibujo del Histograma (III): Ejes, Gradiente y Etiquetas

```

162 |     draw.line([(gx0, gy0), (gx0, gy1), (gx1, gy1)], fill=COLOR_TEXT0, width=2)
165 |     color_ref = series[0][1] if len(series) == 1 else (255, 255, 255)
166 |     for i in range(int(ancho) + 1):
167 |         t = i / ancho
168 |         c = tuple(int(t * comp) for comp in color_ref)
169 |         draw.line([(gx0 + i, gy1 + 4), (gx0 + i, gy1 + 12)], fill=c)
172 |     for nivel in (0, 32, 64, 96, 128, 160, 192, 224, 255):
173 |         x = gx0 + ancho * nivel / 255
174 |         draw.line([(x, gy1 + 12), (x, gy1 + 17)], fill=COLOR_TEXT0, width=1)
175 |         etiqueta = str(nivel)
176 |         tw = draw.textlength(etiqueta, font=FUENTE_TICKS)
177 |         draw.text((x - tw / 2, gy1 + 19), etiqueta, font=FUENTE_TICKS, fill=COLOR_SECUNDARIO)
180 |     etiqueta_x = "Nivel de intensidad"
181 |     tw = draw.textlength(etiqueta_x, font=FUENTE_TEXTO)
182 |     draw.text((gx0 + ancho / 2 - tw / 2, gy1 + 38), etiqueta_x, font=FUENTE_TEXTO, fill=COLOR_TEXT0)
183 |     _texto_vertical(lienzo, "Frecuencia (n.º de píxeles)", (x0 + 20, gy0 + alto / 2),
184 |                    FUENTE_TEXTO, COLOR_TEXT0)

```

Explicación línea por línea:

- **Línea 162:** Dibuja los ejes Y y X como una polilínea en forma de 'L'.
- **Línea 165-169:** Dibuja bajo el eje X una **barra de gradiente** que va de negro al color del canal (o a blanco si hay varias series), indicando visualmente qué intensidad representa cada posición.
- **Línea 172-177:** Coloca marcas y etiquetas del eje X cada 32 niveles (0, 32, ..., 224, 255), centradas bajo cada marca.
- **Línea 180-182:** Escribe el nombre del eje X, centrado.
- **Línea 183-184:** Escribe el nombre del eje Y rotado 90° usando la función auxiliar `_texto_vertical`.

Bloque 10: Panel con la Imagen Original

```

187 | def dibujar_imagen(lienzo, caja, img, titulo, subtitulo=None):
189 |     dibujar_panel(lienzo, caja, titulo, subtitulo)
190 |     x0, y0, x1, y1 = caja
191 |     max_w, max_h = (x1 - x0) - 40, (y1 - y0) - 70
192 |     factor = min(max_w / img.width, max_h / img.height)
193 |     miniatura = img.resize((int(img.width * factor), int(img.height * factor)), Image.LANCZOS)
194 |     px = x0 + ((x1 - x0) - miniatura.width) // 2
195 |     py = y0 + 52 + (max_h - miniatura.height) // 2
196 |     lienzo.paste(miniatura, (px, py))
197 |     ImageDraw.Draw(lienzo).rectangle((px - 1, py - 1, px + miniatura.width, py + miniatura.height),
198 |                                     outline=COLOR_BORDE)

```

Explicación línea por línea:

- **Línea 187-190:** Dibuja el panel base y obtiene sus coordenadas.
- **Línea 191-192:** Calcula el espacio disponible y el factor de escala que permite que la imagen quepa completa **sin deformarse** (se toma el menor de los dos cocientes).
- **Línea 193:** Redimensiona la imagen con el filtro LANCZOS, de alta calidad.
- **Línea 194-196:** Calcula la posición para centrar la miniatura en el panel y la pega sobre el lienzo.
- **Línea 197-198:** Dibuja un marco fino alrededor de la imagen.

Bloque 11: Lienzo General y Encabezado

```

201 | def graficar_histograma(img_rgb, histogramas, estadisticas, ruta_salida=None, mostrar=True):
202 |     ancho_panel, alto_panel, margen, encabezado = 600, 400, 24, 90
203 |     ancho_total = 3 * ancho_panel + 4 * margen
204 |     alto_total = encabezado + 2 * alto_panel + 3 * margen
205 |     lienzo = Image.new("RGB", (ancho_total, alto_total), COLOR_FONDO)
206 |     draw = ImageDraw.Draw(lienzo)
207 |     draw.text((margen, 22), "Ejercicio 2: Histograma de la imagen \"a\" (PIL)",
208 |             font=FUENTE_TITULO, fill=COLOR_TEXTO)
209 |     draw.text((margen, 60), f"Tamaño: {img_rgb.width} x {img_rgb.height} px . "
210 |             font=FUENTE_TEXTO, fill=COLOR_SECUNDARIO)
211 |
212 |     def caja(fila, col):
213 |         x = margen + col * (ancho_panel + margen)
214 |         y = encabezado + margen + fila * (alto_panel + margen)
215 |         return (x, y, x + ancho_panel, y + alto_panel)
216 |
217 |     def sub(canal):
218 |         e = estadisticas[canal]
219 |         return f"μ={e['media']:.1f}  σ={e['desv']:.1f}  med={e['mediana']}"

```

Explicación línea por línea:

- **Línea 201-205:** Define una cuadrícula de 3 columnas x 2 filas de paneles de 600x400 px con márgenes de 24 px y un encabezado de 90 px, y crea el lienzo RGB con el color de fondo.
- **Línea 207-213:** Escribe el título principal y una línea con el tamaño de la imagen, el total de píxeles y el método utilizado.
- **Línea 215-218:** Función interna que calcula la caja (x0, y0, x1, y1) del panel ubicado en una fila y columna dadas.
- **Línea 220-222:** Función interna que arma el subtítulo de cada histograma con la media (mu), la desviación estándar (sigma) y la mediana del canal.

Bloque 12: Distribución de los Seis Paneles y Guardado

```

224 |     dibujar_imagen(lienzo, caja(0, 0), img_rgb, "Imagen original \"a\"")
225 |     dibujar_histograma(lienzo, caja(0, 1), [(histogramas["Gris"], COLORES_CANAL["Gris"])],
226 |                     "Histograma - Escala de grises", sub("Gris"))
227 |     dibujar_histograma(lienzo, caja(0, 2),
228 |                     [(histogramas[c], COLORES_CANAL[c]) for c in ("R", "G", "B")],
229 |                     "Histograma RGB superpuesto", modo="línea")
230 |     for col, canal in enumerate(("R", "G", "B")):
231 |         nombre = {"R": "Rojo (R)", "G": "Verde (G)", "B": "Azul (B)"}[canal]
232 |         dibujar_histograma(lienzo, caja(1, col), [(histogramas[canal], COLORES_CANAL[canal])],
233 |                         f"Canal {nombre}", sub(canal))
234 |
235 |     if ruta_salida:
236 |         lienzo.save(ruta_salida)
237 |         print(f"\nGráfica guardada como: {os.path.basename(ruta_salida)}")
238 |     if mostrar:
239 |         lienzo.show(title="Ejercicio 2 - Histograma (PIL)")
240 |     return lienzo

```

Explicación línea por línea:

- **Línea 224:** Panel (0,0): la imagen original "a".
- **Línea 225-226:** Panel (0,1): histograma en escala de grises, dibujado como barras.
- **Línea 227-229:** Panel (0,2): las tres curvas R, G y B superpuestas en modo línea, para comparar la distribución de los canales.
- **Línea 230-233:** Fila inferior: un histograma de barras por cada canal (rojo, verde, azul), cada uno con su color y sus estadísticas.
- **Línea 235-237:** Guarda el lienzo como PNG si se indicó una ruta.
- **Línea 238-240:** Abre la imagen con el visor predeterminado del sistema y devuelve el lienzo.

Bloque 13: Función Integradora y Programa Principal

```
246 | def histograma_pil(ruta, ruta_salida=None, mostrar=True):
247 |     img = cargar_imagen(ruta)
248 |     print(f"\nImagen: {os.path.basename(ruta)}")
249 |     print(f"  Tamaño: {img.width} x {img.height} píxeles | Modo: {img.mode}")
251 |     histogramas = calcular_histogramas(img)
252 |     estadisticas = calcular_estadisticas(img)
256 |     for canal, hist in histogramas.items():
257 |         e = estadisticas[canal]
258 |         moda = hist.index(max(hist))
262 |     lienzo = graficar_histograma(img, histogramas, estadisticas, ruta_salida, mostrar)
263 |     return histogramas, estadisticas, lienzo
266 | if __name__ == "__main__":
271 |     histograma_pil(RUTA_IMAGEN_A, ruta_salida=RUTA_SALIDA)
```

Explicación línea por línea:

- **Línea 246-249:** Función que encadena todo el proceso: carga la imagen e imprime su nombre, tamaño y modo.
- **Línea 251-252:** Calcula los histogramas y las estadísticas.
- **Línea 254-260:** Imprime una tabla por canal con media, desviación, mediana, mínimo, máximo y la **moda**: el nivel con mayor frecuencia, obtenido con `hist.index(max(hist))`.
- **Línea 262-263:** Genera la gráfica y devuelve todos los resultados.
- **Línea 266-275:** Punto de entrada: imprime el encabezado, ejecuta el análisis de *mono.png* guardando el resultado y muestra el mensaje final.

Resultado obtenido

Ejercicio 2: Histograma de la imagen "a" (PIL)

Tamaño: 318 x 158 px · Total de píxeles: 50244 · Calculado con image.histogram() y dibujado con ImageDraw

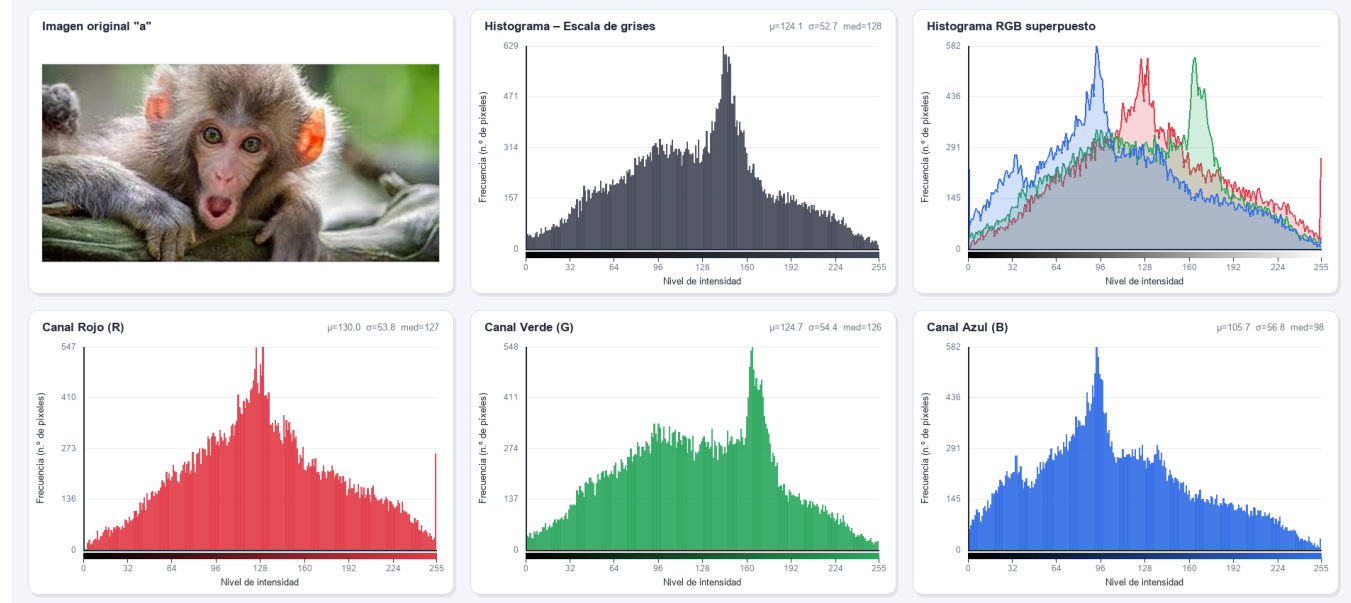


Figura 2. Histogramas en escala de grises y por canal R, G, B generados con PIL (ejercicio_2_resultado.png)

3.3 Ejercicio 3 — Separación de Planos R, G, B y Conversión a Gris

Archivo fuente: `ejercicio_3.py` (Código limpio sin comentarios)

Bloque 1: Importaciones, Rutas y Constantes

```
1 | import os
3 | import cv2
4 | import numpy as np
5 | import matplotlib.pyplot as plt
7 | directorio = os.path.dirname(os.path.abspath(__file__))
12 | RUTA_IMAGEN_B = os.path.join(directorio, "flores.png")
13 | RUTA_SALIDA_CANALES = os.path.join(directorio, "ejercicio_3_canales.png")
14 | RUTA_SALIDA_GRIS = os.path.join(directorio, "ejercicio_3_gris.png")
15 | RUTA_IMAGEN_GRIS = os.path.join(directorio, "flores_gris.png")
17 | NOMBRES_CANALES = ("Rojo (R)", "Verde (G)", "Azul (B)")
18 | COLORES_HIST = ("#e23d4a", "#2ea860", "#346ee6")
```

Explicación línea por línea:

- **Línea 1-5:** Importa `os`, `OpenCV`, `NumPy` y `Matplotlib`.
- **Línea 7:** Directorio del script, base para las rutas.
- **Línea 12:** Ruta de la imagen de entrada `"b" (flores.png)`.
- **Línea 13-15:** Rutas de salida: figura de canales, figura comparativa de grises y la imagen convertida a gris en sí.
- **Línea 17-18:** Tuplas con los nombres de los canales y los colores hexadecimales usados en los histogramas, en el orden R, G, B.

Bloque 2: Carga de la Imagen, Separación de Canales y Canal Teñido

```
21 | def cargar_imagen_rgb(ruta):
22 |     img_bgr = cv2.imread(ruta, cv2.IMREAD_COLOR)
23 |     if img_bgr is None:
24 |         raise FileNotFoundError(f"No se pudo cargar la imagen: {ruta}")
25 |     return cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
28 | def separar_canales(img_rgb):
29 |     r, g, b = cv2.split(img_rgb)
30 |     return r, g, b
33 | def canal_coloreado(canal, indice):
34 |     img = np.zeros((*canal.shape, 3), dtype=np.uint8)
35 |     img[:, :, indice] = canal
36 |     return img
```

Explicación línea por línea:

- **Línea 21-24:** Lee la imagen en color y verifica que se haya cargado correctamente.
- **Línea 25:** `OpenCV` almacena los canales en orden **BGR**; se convierten a **RGB** para que la separación y la visualización con `Matplotlib` sean correctas.
- **Línea 28-30:** `cv2.split` divide la matriz (alto x ancho x 3) en tres matrices 2D independientes, una por canal, con valores de 0 a 255.
- **Línea 33-34:** Crea una imagen RGB completamente negra del mismo tamaño que el canal (`*canal.shape` desempaqueta alto y ancho).
- **Línea 35-36:** Copia el canal solo en su posición (0 = R, 1 = G, 2 = B), dejando los otros dos en 0. Así se visualiza el canal 'teñido' con su color puro.

Bloque 3: Conversión a Escala de Grises (OpenCV y Manual)

```

39 | def convertir_a_gris(img_rgb):
40 |     gris_opencv = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
42 |     r, g, b = [c.astype(np.float64) for c in separar_canales(img_rgb)]
43 |     gris_manual = np.clip(np.round(0.299 * r + 0.587 * g + 0.114 * b), 0, 255).astype(np.uint8)
45 |     return gris_opencv, gris_manual

```

Explicación línea por línea:

- **Línea 40:** Método 1: conversión nativa de OpenCV con COLOR_RGB2GRAY.
- **Línea 42:** Método 2 (manual): separa los canales y los convierte a coma flotante para evitar el desbordamiento de uint8 al multiplicar y sumar.
- **Línea 43:** Aplica la fórmula de luminancia ITU-R BT.601: $Y = 0.299R + 0.587G + 0.114B$. Los pesos reflejan la sensibilidad del ojo humano (mayor al verde, menor al azul). El resultado se redondea, se limita a [0, 255] con np.clip y se convierte a uint8.
- **Línea 45:** Devuelve ambas versiones para compararlas.

Bloque 4: Figura de Canales (I): Imagen Original e Histogramas

```

48 | def graficar_canales(img_rgb, canales, ruta_salida=None):
49 |     fig, axes = plt.subplots(2, 4, figsize=(20, 9))
50 |     fig.suptitle("Ejercicio 3: Separación de los planos de color R, G, B",
51 |                 fontsize=16, fontweight="bold")
54 |     axes[0, 0].imshow(img_rgb)
55 |     axes[0, 0].set_title("Imagen original \"b\" (RGB)", fontsize=12, fontweight="bold")
56 |     axes[0, 0].axis("off")
59 |     max_interno = 0
60 |     for canal, nombre, color in zip(canales, NOMBRES_CANALES, COLORES_HIST):
61 |         hist = cv2.calcHist([canal], [0], None, [256], [0, 256]).ravel()
62 |         max_interno = max(max_interno, hist[1:255].max())
63 |         axes[1, 0].plot(hist, color=color, label=nombre, linewidth=1.4)
64 |         axes[1, 0].fill_between(range(256), hist, color=color, alpha=0.15)
65 |         axes[1, 0].set_title("Histograma por canal", fontsize=12, fontweight="bold")
66 |         axes[1, 0].set_xlim(0, 255)
69 |         axes[1, 0].set_ylim(0, max_interno * 1.15)
70 |         axes[1, 0].set_xlabel("Nivel de intensidad")
71 |         axes[1, 0].set_ylabel("Frecuencia")
72 |         axes[1, 0].legend()
73 |         axes[1, 0].grid(alpha=0.3)

```

Explicación línea por línea:

- **Línea 48-51:** Crea una figura de 2 filas x 4 columnas con su título general.
- **Línea 54-56:** Panel (0,0): imagen original sin ejes.
- **Línea 59-61:** Para cada canal calcula su histograma con cv2.calcHist (256 bins en el rango [0, 256]); ravel() lo convierte de (256, 1) a un vector 1D.
- **Línea 62:** Guarda el máximo de frecuencia **excluyendo los niveles 0 y 255**. Esos extremos suelen tener picos enormes por zonas saturadas o muy oscuras, que aplastarían el resto del histograma.
- **Línea 63-64:** Dibuja la curva de cada canal con su color y un relleno semitransparente (alpha = 0.15).
- **Línea 65-66:** Título del panel y rango del eje X de 0 a 255.
- **Línea 69:** Limita el eje Y al máximo interno + 15%, de modo que la forma de la distribución se aprecie con claridad.
- **Línea 70-73:** Etiquetas de los ejes, leyenda con los nombres de los canales y grilla tenue.

Bloque 5: Figura de Canales (II): Planos en Intensidad y Teñidos

```

76 |     for i, (canal, nombre) in enumerate(zip(canales, NOMBRES_CANALES)):
77 |         axes[0, i + 1].imshow(canal, cmap="gray", vmin=0, vmax=255)
78 |         axes[0, i + 1].set_title(f"Canal {nombre} - intensidad\n"
79 |                                 f"media = {canal.mean():.1f}", fontsize=11)
80 |
81 |         axes[0, i + 1].axis("off")
82 |
83 |         axes[1, i + 1].imshow(canal_coloreado(canal, i))
84 |         axes[1, i + 1].set_title(f"Canal {nombre} - teñido", fontsize=11)
85 |         axes[1, i + 1].axis("off")
86 |
87 | plt.tight_layout(rect=[0, 0, 1, 0.95])
88 | if ruta_salida:
89 |     fig.savefig(ruta_salida, dpi=150, bbox_inches="tight")
90 |     print(f" Figura de canales guardada como: {os.path.basename(ruta_salida)}")
91 |
92 | return fig

```

Explicación línea por línea:

- **Línea 76:** Recorre los tres canales con su índice (0, 1, 2) para ubicarlos en las columnas 1 a 3.
- **Línea 78-81:** Fila superior: cada canal como imagen en escala de grises. Se fija `vmin=0`, `vmax=255` para que el brillo sea absoluto y comparable entre canales (sin autoescalado). El título incluye la intensidad media.
- **Línea 84-86:** Fila inferior: cada canal 'teñido' con su color puro mediante `canal_coloreado`.
- **Línea 88-92:** Ajusta el diseño, guarda la figura a 150 DPI y la devuelve.

Bloque 6: Figura Comparativa de la Conversión a Gris

```

95 | def graficar_gris(img_rgb, gris_opencv, gris_manual, ruta_salida=None):
96 |     diferencia = cv2.absdiff(gris_opencv, gris_manual)
97 |     fig, axes = plt.subplots(1, 4, figsize=(20, 5))
98 |     fig.suptitle("Ejercicio 3: Conversión a escala de grises", fontsize=16, fontweight="bold")
99 |     axes[0].imshow(img_rgb)
100 |
101 |     axes[1].imshow(gris_opencv, cmap="gray", vmin=0, vmax=255)
102 |     axes[2].imshow(gris_manual, cmap="gray", vmin=0, vmax=255)
103 |     im = axes[3].imshow(diferencia, cmap="magma", vmin=0, vmax=max(1, int(diferencia.max())))
104 |     axes[3].set_title(f"|OpenCV - Manual| (máx = {diferencia.max():.1f})", fontsize=12)
105 |     fig.colorbar(im, ax=axes[3], fraction=0.035, pad=0.02)
106 |     for ax in axes:
107 |         ax.axis("off")
108 |
109 |     plt.tight_layout(rect=[0, 0, 1, 0.92])
110 |     if ruta_salida:
111 |         fig.savefig(ruta_salida, dpi=150, bbox_inches="tight")
112 |
113 |     return fig

```

Explicación línea por línea:

- **Línea 95-96:** Calcula la diferencia absoluta píxel a píxel entre ambos métodos con `cv2.absdiff` (evita valores negativos en `uint8`).
- **Línea 98-99:** Crea una figura de 1 fila x 4 paneles.
- **Línea 101-108:** Muestra la imagen original, el gris de OpenCV y el gris manual, ambos con la escala fija 0-255.
- **Línea 110-112:** Muestra el mapa de diferencias con el colormap 'magma' y una barra de color. El `max(1, ...)` evita un rango nulo si ambas imágenes son idénticas. Las diferencias esperadas son de 0 o 1 nivel, debidas solo al redondeo interno de OpenCV.
- **Línea 114-115:** Oculta los ejes de todos los paneles.
- **Línea 117-121:** Ajusta el diseño, guarda la figura y la devuelve.

Bloque 7: Función Integradora y Programa Principal

```

124 | def procesar_imagen_b(ruta, mostrar=True):
125 |     img_rgb = cargar_imagen_rgb(ruta)
126 |     alto, ancho, _ = img_rgb.shape
132 |     canales = separar_canales(img_rgb)
133 |     for canal, nombre in zip(canales, NOMBRES_CANALES):
134 |         print(f" {nombre:<10} forma={canal.shape} media={canal.mean():7.2f}  "
135 |               f"mín={canal.min():3d} máx={canal.max():3d}")
136 |     graficar_canales(img_rgb, canales, RUTA_SALIDA_CANALES)
140 |     gris_opencv, gris_manual = convertir_a_gris(img_rgb)
143 |     print(f" Diferencia máxima entre métodos: {cv2.absdiff(gris_opencv, gris_manual).max()}")
144 |     cv2.imwrite(RUTA_IMAGEN_GRIS, gris_opencv)
146 |     graficar_gris(img_rgb, gris_opencv, gris_manual, RUTA_SALIDA_GRIS)
148 |     if mostrar:
149 |         plt.show()
151 |     return {"canales": canales, "gris_opencv": gris_opencv, "gris_manual": gris_manual}
154 | if __name__ == "__main__":
159 |     procesar_imagen_b(RUTA_IMAGEN_B)

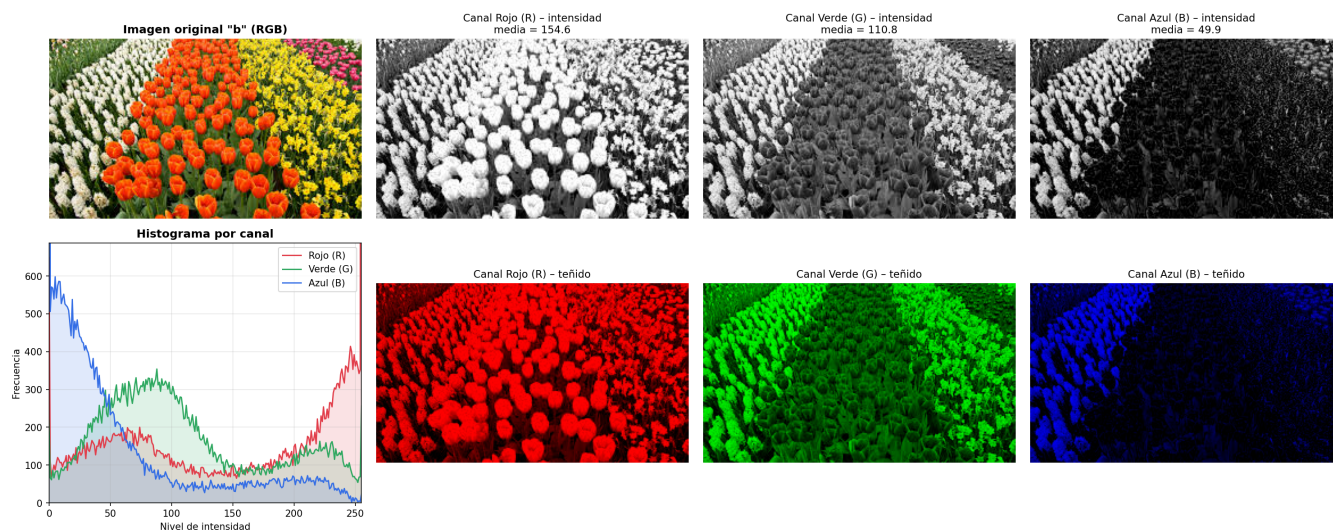
```

Explicación línea por línea:

- **Línea 124-128:** Carga la imagen "b", obtiene sus dimensiones desde shape (alto, ancho, canales) y las imprime.
- **Línea 131-135:** Separa los canales e imprime, para cada uno, su forma, intensidad media, mínimo y máximo.
- **Línea 136:** Genera y guarda la figura de separación de canales.
- **Línea 139-143:** Convierte la imagen a gris por ambos métodos e imprime la media de cada resultado y la diferencia máxima entre ellos.
- **Línea 144-145:** Guarda la imagen en escala de grises como *flores_gris.png*; imwrite acepta directamente una matriz de un solo canal.
- **Línea 146:** Genera la figura comparativa de la conversión.
- **Línea 148-151:** Muestra las ventanas de Matplotlib y devuelve los resultados.
- **Línea 154-163:** Punto de entrada: imprime el encabezado, procesa *flores.png* y muestra el mensaje final.

Resultado obtenido

Ejercicio 3: Separación de los planos de color R, G, B



Ejercicio 3: Conversión a escala de grises

